Компьютерная графика

Домашнее задание 2: Визуализатор сцены с освещением и тенями

2026

Задание

- Сделать визуализатор сцен в формате OBJ с текстурами и освещением по Фонгу

Задание

- Сделать визуализатор сцен в формате OBJ с текстурами и освещением по Фонгу
- Путь до сцены задаётся, например, параметром командной строки

Задание

- Сделать визуализатор сцен в формате OBJ с текстурами и освещением по Фонгу
- Путь до сцены задаётся, например, параметром командной строки
- Два источника света: направленный (‘солнце’) и точечный, оба как-то двигаются со временем

Задание

- Сделать визуализатор сцен в формате OBJ с текстурами и освещением по Фонгу
- Путь до сцены задаётся, например, параметром командной строки
- Два источника света: направленный (‘солнце’) и точечный, оба как-то двигаются со временем
- От обоих источников света есть тени, построенные алгоритмом shadow mapping (+ PCF или VSM, с размытием)

Задание

- Сделать визуализатор сцен в формате OBJ с текстурами и освещением по Фонгу
- Путь до сцены задаётся, например, параметром командной строки
- Два источника света: направленный (‘солнце’) и точечный, оба как-то двигаются со временем
- От обоих источников света есть тени, построенные алгоритмом shadow mapping (+ PCF или VSM, с размытием)
- Камера должна управляться пользователем (например, так же, как в первом домашнем задании)

Сцена

Хорошая сцена:

- Порядка 100k-1kk треугольников в сумме
- Порядка 100-1000 различных объектов
- У большинства объектов есть текстура альбедо

Рекомендую тестировать на *Crytek Sponza*, её можно найти здесь:
`casual-effects.com/data/index.html`

Crytek Sponza



Crytek Sponza



Crytek Sponza

- Одна из самых популярных тестовых сцен в 3D графике
- Модель атриума дворца Спонца в Дубровнике (Хорватия)

Сцена: формат OBJ

Wavefront OBJ – один из общепринятых форматов сцен

- Текстовый, достаточно простой для чтения
- Содержит координаты вершин, нормали и текстурные координаты
- Содержит индексы вершин, образующих треугольники
- Может содержать много объектов
- Может ссылаться на MTL-файл (Material Template Library), содержащий описания материалов
- MTL может в свою очередь ссылаться на текстуры (альбедо, нормали, и т.п.)

Сцена: формат OBJ

Что нужно из описания материалов:

- Текстура альбедо `map_Ka` – коэффициент для диффузного освещения (без нормализации; specular компоненту на неё **не умножаем**)
- Тестура прозрачности `map_d` – делаем discard, если в текстуре значение меньше 0.5 (нужно использовать в основном шейдере и в шейдере для shadow map), блендинг делать **не нужно**
- Коэффициент отражения (glossiness) `Ks` – множитель для мощности specular компоненты
- Показатель отражения (power) `Ns` – показатель степени для specular компоненты
- Specular компоненту используем с нормализацией

Сцена: формат OBJ

- Можно написать загрузчик руками или дописать тот, что есть у нас в практиках
- Можно использовать какую-нибудь библиотеку, например Assimp или TinyOBJ
- **N.B.** Есть два соглашения о том, как идут текстурные координаты: снизу вверх, или сверху вниз. Если текстуры будут выглядеть странно, попробуйте отразить текстурные координаты по оси Y: `y = 1 - y`

Советы

- Можно иметь один общий набор VAO + VBO + EBO, и для каждого объекта хранить только диапазон индексов
- Можно иметь по VAO + VBO + EBO на каждый объект
- Скорее всего, у вас будет 1 draw call (`glDrawElements` или т.п.) на один объект
- Напоминание: для рендеринга VBO и EBO не нужны, только `glBindVertexArray` и `glDraw*`!

Советы

- В сумме у вас будет 2 shadow map (одна обычная, для направленного источника; вторая – cubemap, для точечного)
- В сумме у вас будет 8 texture view для shadow map (одна для направленного источника; одна для семплинга cubemap текстуры в основном шейдере; 6 для генерации граней cubemap текстуры)
- Размер shadow map лучше взять побольше, если не будет тормозить (4096x4096)
- Размывать тени можно прямо в результирующем шейдере, при чтении из shadow map, а можно – в два прохода, сепарабельным фильтром за $O(N)$ *(см. separable gaussian blur)*, за это будет больше баллов
- Радиус размытия – как можно больше, пока не начинает тормозить :)

Советы

- При загрузке полезно посчитать bounding box сцены, чтобы потом по нему вычислять матрицы для теней
- Лучше вычислять проекцию тени для направленного источника света используя **пересечение** bounding box'а видимой области камеры (см. конец лекции 4) и bounding box'а самой сцены, чтобы ничего лишнего не попало в shadow map, за это будет больше баллов (**NB:** для этого нужно включить *depth clamping*)
- При инициализации wgpu_app (и в C++, и в Rust) **нужно** передавать `srgb = false` (что это значит, мы узнаем позже)

Баллы

- **1 балл**: движется камера
- **1 балл**: геометрия сцены загружается и рисуется
- **1 балл**: есть текстуры альбедо
- **1 балл**: есть текстуры прозрачности
- **1 балл**: есть ambient освещение + два движущихся источника света
- **2 балла**: есть тени от направленного источника света
  - **+1 балл**: PCF + blur
  - либо **+1 балл**: VSM + blur
  - либо +**2 балла**: VSM + separable blur
- **1 балл**: проеция тени для направленного источника вычисляется на основе camera frustum
- **3 балла**: есть тени от точечного источника света
  - **+1 балл**: PCF + blur
  - либо **+1 балл**: VSM + blur
  - либо **+2 балла**: VSM + separable blur

Всего: **15 баллов**

Защита заданий на практике **через 2 недели**.